

Parallel and Distributed Systems: Paradigms and Models

Project 1 (MergeSort) Report

Davide Marchi (602476)
`d.marchi5@studenti.unipi.it`

August 2025

Contents

1	Introduction	2
2	Problem description	2
2.1	Data model and constraints	2
2.2	Pipeline, distributed setting, and metrics	3

1 Introduction

I present a complete study of a distributed out-of-core MergeSort. The study proceeds in four steps. First, I describe the problem and the data layout. Second, I implement four solutions that share a common I/O pipeline while differing in how they exploit parallelism: a sequential baseline, an OpenMP task based version, a FastFlow farm based version, and a multi node hybrid MPI solution that reuses the single node core. Third, I evaluate these versions under controlled workloads and hardware configurations, measuring runtime, speedup, and efficiency. Finally, I discuss the results and propose an approximate cost model for the distributed solution.

All implementations work on files that are potentially larger than available RAM. Each program first scans the unsorted file to build an in memory index of keys and positions and lengths, then sorts this index, then rewrites a new file in key order by fetching payloads at the recorded positions. To keep memory usage proportional to the number of records, the in memory structure stores only keys, offsets, and lengths. The write phase can be highly variable on a shared cluster, so the primary performance analysis focuses on the reading and sorting stages. The full pipeline remains functionally correct and the final output is validated.

The objectives are:

- demonstrate correctness of all variants by verifying that keys are globally sorted in the *written* output file and that records are preserved;
- quantify single node scalability with strong and weak scaling for OpenMP and FastFlow;
- quantify multi node scalability for the MPI hybrid, using one rank per node and varying threads per rank;
- interpret bottlenecks that arise from memory access, tasking overheads, and communication.

Structure. Section 2 defines the problem and data model. Section 3 discusses memory and I/O considerations. Section 4 presents single node implementations and their evaluation. Section 5 presents the multi node implementation and its evaluation. Section 6 proposes an approximate cost model and discusses bottlenecks and overlap.

2 Problem description

2.1 Data model and constraints

The input is a large binary file that stores an array of records. Each record consists of a sortable key, a length field, and a payload. The payload size can be large compared to the key, and the file may exceed available RAM, which motivates an out-of-core strategy.

I use the following logical layout for each **Record**:

- **key**: 64 bit unsigned integer used for ordering;
- **len**: 32 bit unsigned integer that gives the payload length in bytes, with $\text{len} \leq P$ for a dataset parameter P ;
- **payload**: a byte array of length len .

Sorting records in place is impractical when the file is larger than RAM. Instead, I construct an auxiliary in memory index. Each entry is an **IndexRec** that stores only what is needed to order and later locate the record:

- **key**: 64 bit unsigned integer, copied from the record;
- **pos**: byte offset in the unsorted file where the record begins;
- **len**: 32 bit unsigned integer, copied from the record, used to stream exactly len bytes during rewrite.

The index has length N , one entry per record. Its memory footprint is $O(N)$ and is independent of the maximum payload size P . This design keeps memory predictable and minimizes disk traffic during random access.

Correctness means that the output file contains exactly the input records in nondecreasing key order. I validate this by scanning the *written* output file from disk, checking that keys are ordered and that the record count matches the input.

2.2 Pipeline, distributed setting, and metrics

The pipeline is the same across all variants:

1. **Index build.** Scan the unsorted file, read **key** and **len** per record, and fill the **IndexRec** array with $\{\text{key}, \text{pos}, \text{len}\}$. File access uses memory mapping to support efficient random access patterns and to avoid large user space buffers.
2. **Index sort.** Sort the **IndexRec** array by key. The sequential baseline uses `std::sort`. The parallel versions use a task based mergesort with a base case threshold that delegates small subarrays to `std::sort`.
3. **Rewrite.** Create the output file, then for each entry in sorted order fetch the record at **pos** from the unsorted file and write it to the next position in the output file, streaming exactly **len** bytes for the payload. This preserves the original payloads without storing them all in memory.
4. **Validation.** Scan the output file to confirm nondecreasing key order and that the record count is unchanged.

In the multi node variant, I run one MPI rank per node. A designated rank reads the file and builds the global index, then partitions that index into contiguous slices and sends one slice to each rank. Each rank sorts its slice locally. The globally sorted order is

obtained through a sequence of pairwise merges across ranks. The final rank rewrites the output file and performs validation. Communication is limited to index data and merged index segments, which keeps memory usage predictable and decoupled from payload size.

The primary metrics are total time for index build plus sort, speedup $S_p = T_1/T_p$, and efficiency $E_p = S_p/p$. For single node studies, p is the thread count. For distributed studies, p is the product of nodes and threads if noted, or the number of threads per rank when speedup is plotted against nodes. Strong scaling holds N fixed while varying p . Weak scaling grows N proportionally to p .